

Code Optimization: Space Complexity (with Generative AI)

This section complements your **Time Complexity** module and completes the performance optimization topic.

Understanding Space Complexity

Space Complexity measures how much memory an algorithm uses as the input size increases.

Includes:

- Input storage
- Auxiliary (temporary) memory
- Call stack (recursion)

Measured using **Big-O notation**.

Common Space Complexities

Big-O	Meaning	Example
O(1)	Constant	Swapping variables
O(n)	Linear	Array copy
O(n²)	Quadratic	2D matrix
O(log n)	Logarithmic	Recursive binary search

Why Space Complexity Matters

- Reduces memory consumption
 - Improves speed
 - Lowers cloud costs
 - Supports large datasets
-

Optimize Space Complexity with Generative AI

Generative AI tools:

- Identify unnecessary variables
- Suggest in-place operations
- Replace large structures with efficient ones
- Convert recursion → iteration

How Generative AI Helps

Optimization Area	AI Assistance
Remove extra arrays	Yes
Suggest in-place sorting	Yes
Replace recursion	Yes
Choose efficient data structures	Yes

Example 1: Removing Extra Array

Before ($O(n)$ extra space)

```
def square_numbers(nums):
    result = []
    for n in nums:
        result.append(n*n)
    return result
```

After (In-place modification)

```
def square_numbers(nums):
    for i in range(len(nums)):
        nums[i] = nums[i] * nums[i]
    return nums
```

Example 2: Recursive → Iterative

Before ($O(n)$ stack)

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n-1)
```

After ($O(1)$ space)

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```

Example 3: Using Generator Instead of List

Before

```
nums = [i for i in range(1000000)]
```

After

```
nums = (i for i in range(1000000))
```

Space Complexity with Generative AI: Demo Workflow

1. Paste code into IDE
2. Add comment:

```
# Optimize space complexity
```

3. Copilot suggests in-place solution
 4. Review and test
-

Demo: Improving Space Complexity Using Generative AI

Input Code

```
def copy_list(nums):  
    new_list = []  
    for n in nums:  
        new_list.append(n)  
    return new_list
```

Copilot Suggestion

```
def copy_list(nums):  
    return nums[:] # Shallow copy (simpler & efficient)
```

or

```
def process_list(nums):  
    for i in range(len(nums)):  
        nums[i] += 1  
    return nums
```

Best Practices

- Prefer in-place operations
 - Avoid unnecessary copies
 - Balance readability
-

Limitations

- In-place changes may break logic
- AI may suggest unsafe memory reuse

Always validate.

Quick Check

Q1. What is space complexity?

Memory used by algorithm

Q2. True/False

Generators use less memory than lists.

True

Q3. Which is better for space?

- A) Extra array
- B) In-place update

B

Summary

Generative AI assists developers in:

- ✓ Reducing memory usage
- ✓ Suggesting in-place operations
- ✓ Improving scalability

But **human review is essential.**
